

EXPERIMENT 5

Student Name: Husanpreet Kaur

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70138

Section/Group: 24AIT_KRG-G2

Date: 10-08-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, self-joins, and date operations to solve real-world database problems.

Question 1:

(i) (A) A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.

For each registered user, calculate the confirmation rate, which is defined as:

$$\text{Confirmation Rate} = \frac{\text{Number of Successful Confirmations}}{\text{Total Confirmation Requests}}$$

If a user has never received any confirmation request, their confirmation rate should be considered 0.

Display the user ID along with the confirmation rate rounded to 2 decimal places. The result can be returned in any order.

Table:

CREATE TABLE Signups

(

EXPERIMENT 5

```

user_id INT PRIMARY KEY,
time_stamp TIMESTAMP
);

```

```

INSERT INTO Signups
VALUES
(3,'2020-03-21 10:16:13'),
(7,'2020-01-04 13:57:59'),
(2,'2020-07-29 23:09:44'),
(6,'2020-12-09 10:39:37');

```

```

SELECT * FROM Signups;

```

Data Output			Messages	Notifications
				
			SQL	
	user_id [PK] integer	time_stamp timestamp without time zone		
1	3	2020-03-21 10:16:13		
2	7	2020-01-04 13:57:59		
3	2	2020-07-29 23:09:44		
4	6	2020-12-09 10:39:37		

```

CREATE TABLE Confirmations
(

```

```

    user_id INT,
    time_stamp TIMESTAMP,
    action VARCHAR(20),
    PRIMARY KEY(user_id, time_stamp),
    FOREIGN KEY(user_id) REFERENCES Signups(user_id)
);

```

```

INSERT INTO Confirmations
VALUES
(3,'2021-01-06 03:30:46','timeout'),

```

EXPERIMENT 5

```
(3,'2021-07-14 14:00:00','timeout'),
(7,'2021-06-12 11:57:29','confirmed'),
(7,'2021-06-13 12:58:28','confirmed'),
(7,'2021-06-14 13:59:27','confirmed'),
(2,'2021-01-22 00:00:00','confirmed'),
(2,'2021-02-28 23:59:59','timeout');
```

```
SELECT * FROM Confirmations;
```

Data Output Messages Notifications			
	user_id [PK] integer	time_stamp [PK] timestamp without time zone	action character varying (20)
1	3	2021-01-06 03:30:46	timeout
2	3	2021-07-14 14:00:00	timeout
3	7	2021-06-12 11:57:29	confirmed
4	7	2021-06-13 12:58:28	confirmed
5	7	2021-06-14 13:59:27	confirmed
6	2	2021-01-22 00:00:00	confirmed
7	2	2021-02-28 23:59:59	timeout

Answer: SELECT

```
s.user_id,
ROUND(
  AVG(
    CASE
      WHEN c.action = 'confirmed' THEN 1.0
      ELSE 0.0
    END
  ), 2
) AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c
  ON s.user_id = c.user_id
```

EXPERIMENT 5

GROUP BY s.user_id;

Output:

Data Output	Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 📊 ⬇️ 📈 SQL		
	user_id [PK] integer	confirmation_rate numeric
1	2	0.50
2	6	0.00
3	3	0.00
4	7	1.00

Question 2: A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between 2013-10-01 and 2013-10-03, determine the proportion of canceled trips among all valid trips. A trip should be considered valid only when both the customer and the driver associated with that trip are active users. The final result should display the date and the calculated cancellation rate rounded to two decimal places.

We need to calculate:

Cancellation Rate

=

Cancelled Trips

Total Valid Trips

But only consider trips where: Client is NOT banned

AND

Driver is NOT banned

EXPERIMENT 5

And Date between

2013-10-01

And

2013-10-03

Table: CREATE TABLE Users

```
(  
    users_id INT PRIMARY KEY,  
    banned VARCHAR(3),  
    role VARCHAR(20)  
);
```

INSERT INTO Users

VALUES

```
(1,'No','client'),  
(2,'Yes','client'),  
(3,'No','client'),  
(4,'No','client'),  
(10,'No','driver'),  
(11,'No','driver'),  
(12,'No','driver'),  
(13,'No','driver');
```

```
select * from Users;
```

EXPERIMENT 5

Data Output

Messages

Notifications

SQL

	users_id [PK] integer	banned character varying (3)	role character varying (20)
1	1	No	client
2	2	Yes	client
3	3	No	client
4	4	No	client
5	10	No	driver
6	11	No	driver
7	12	No	driver
8	13	No	driver

CREATE TABLE Trips

```
(
  id INT PRIMARY KEY,
  client_id INT,
  driver_id INT,
  city_id INT,
  status VARCHAR(30),
  request_at DATE
);
```

INSERT INTO Trips

VALUES

```
(1,1,10,1,'completed','2013-10-01'),
(2,2,11,1,'cancelled_by_driver','2013-10-01'),
(3,3,12,6,'completed','2013-10-01'),
(4,4,13,6,'cancelled_by_client','2013-10-01'),
(5,1,10,1,'completed','2013-10-02'),
(6,2,11,6,'completed','2013-10-02'),
(7,3,12,6,'completed','2013-10-02'),
(8,2,12,12,'completed','2013-10-03'),
```

EXPERIMENT 5

(9,3,10,12,'completed','2013-10-03'),
 (10,4,13,12,'cancelled_by_driver','2013-10-03');

select * from Trips;

Data Output

Messages

Notifications

≡

+

SQL

Show

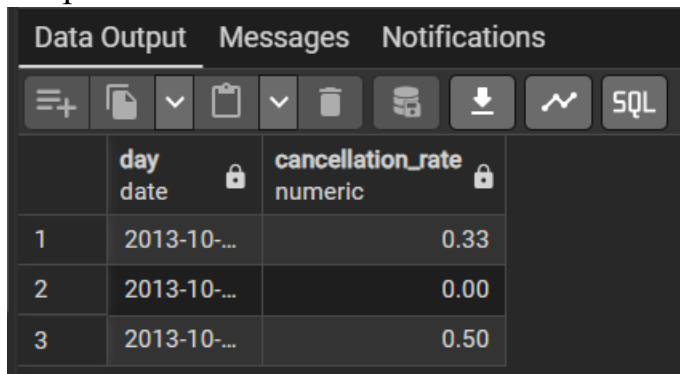
	id [PK] integer	client_id integer	driver_id integer	city_id integer	status character varying (30)	request_at date
1	1	1	10	1	completed	2013-10-01
2	2	2	11	1	caelled_by_driver	2013-10-01
3	3	3	12	6	completed	2013-10-01
4	4	4	13	6	cancelled_by_client	2013-10-01
5	5	1	10	1	completed	2013-10-02
6	6	2	11	6	completed	2013-10-02
7	7	3	12	6	completed	2013-10-02
8	8	2	12	12	completed	2013-10-03
9	9	3	10	12	completed	2013-10-03
10	10	4	13	12	cancelled_by_driver	2013-10-03

Answer: SELECT
 t.request_at AS Day,
 ROUND(
 AVG(
 CASE
 WHEN t.status IN ('cancelled_by_driver', 'cancelled_by_client')
 THEN 1.0
 ELSE 0.0
 END
),
 2
) AS Cancellation_Rate
 FROM Trips t
 JOIN Users c

EXPERIMENT 5

```
ON t.client_id = c.users_id
AND c.banned = 'No'
AND t.request_at between '2013-10-01' and '2013-10-03'
JOIN Users d
  ON t.driver_id = d.users_id
  AND d.banned = 'No'
AND t.request_at between '2013-10-01' and '2013-10-03'
GROUP BY t.request_at
ORDER BY t.request_at;
```

Output:



	day date	cancellation_rate numeric
1	2013-10-...	0.33
2	2013-10-...	0.00
3	2013-10-...	0.50

LEARNING OUTCOMES:

1. Learned to calculate confirmation rates using aggregate functions such as AVG() and conditional logic with the CASE statement.
2. Gained practical understanding of using LEFT JOIN to include all registered users, even those who have not received any confirmation requests.
3. Learned to use self-joins on the Users table to separately identify and validate active clients and drivers in trip records.
4. Developed the ability to calculate cancellation rates using conditional aggregation, AVG(), and ROUND() for accurate two-decimal results.
5. Enhanced understanding of combining JOIN, WHERE/date conditions, GROUP BY, ORDER BY, CASE, and aggregate functions to solve real-world database problems involving user confirmations and ride cancellations.